Reliability Engineering for Field-Deployed ESP32 Sensor Nodes: A Case Study in Fault Detection, Recovery, and Bounded Latency

Baxrom R. Reyimberganov[1,*]

[1]Department of Software Engineering, Urgench State University, Urgench, 220100, Uzbekistan

*Corresponding author: Baxrom R. Reyimberganov, Email: bahromreyimberganov0311@gmail.com, ORCID: 0009-0005-8124-0042

Keywords: embedded systems reliability, watchdog timers, fault detection, sensor calibration, ESP32, fail-safe design

Abstract

Field-deployed embedded sensor nodes fail silently far more often than they fail loudly: a disconnected probe, a momentarily unresponsive I2C peripheral, or a Wi-Fi access point that is slower to boot than the node itself do not, by default, produce an error — they produce a plausible but wrong number, or a frozen display indistinguishable from a live one. This paper reports a systematic reliability audit of an ESP32-based multi-sensor firmware fleet (32 build configurations across six utility types) and the fixes applied. We document sixteen distinct defects spanning four categories — unbounded blocking with no watchdog protection, sensor fault masking, unbounded memory allocation, and boot-time race conditions — present the calibration and validity-check formulas used to distinguish a genuine reading from a rail-pinned sensor, and report a compile-matrix verification methodology (fifteen build configurations, full recompilation after every change) used as a regression gate throughout. We conclude with a case where a proposed fix (a receive-side checksum for a serial metering protocol) was deliberately reverted after code review because it could not be validated against physical hardware, illustrating a conservative-by-default policy for firmware that has not yet been field tested.

1 Introduction

An embedded sensor node that is unreachable produces an obvious signal: its absence. An embedded sensor node that is *malfunctioning while appearing to work* is far more dangerous operationally, because nothing in the system — neither the node itself, nor the backend it reports to, nor the human glancing at a wall-mounted display — has any reason to distrust the number it is looking at. This failure mode is the central concern of this paper: not "does the device crash," but "when the device or the physical world around it degrades, does the system notice?"

We report on a systematic reliability audit of the firmware fleet underlying the platform described in a companion architecture paper ("A Layered Architecture for Multi-Utility IoT Monitoring"): 32 distinct PlatformIO build configurations, covering six utility types (electricity, water, gas, soil moisture, sound, heating) across three deployment roles (standalone WiFi node, RS-485 bus leaf, RS-485 bridge/collector). The audit proceeded in four passes of increasing depth — code-level sensor logic, RS-485 bus/bridge timing, ESP32 platform-level stability primitives (watchdog, brownout, heap), and finally the failure

modes introduced by the fixes from the earlier passes themselves — and is, to our knowledge, a reasonably complete account of the defect classes one should expect to find in a field firmware fleet that grew organically over multiple sensor types without a single reliability review pass.

Our contributions are: (1) a taxonomy of four defect classes recurring across otherwise-unrelated sensor drivers and subsystems; (2) the specific validity-check formulas used to convert a raw ADC or protocol reading into a *trustworthy-or-flagged* value, derived from each sensor's own calibration; (3) a report of a compile-matrix regression methodology used to keep sixteen sequential fixes from silently breaking any of the fleet's 32 build targets; and (4) an explicit account of a fix that was implemented, verified to compile, and then *deliberately reverted* after reasoning about the asymmetry between its potential benefit and its unverified risk — offered as a worked example of when "it compiles and looks correct" is not sufficient justification to ship embedded code that has not been exercised against real hardware.

2 METHODS

2.1 Audit methodology

Each pass targeted a different layer of the system and used the same discipline: read the full source of the file(s) under review (not a diff), trace every code path including error branches, and — for any candidate fix — recompile the full affected subset of the 32 build configurations via PlatformIO before considering the fix complete. Table 1 summarizes the four passes.

Pass	Scope
1	Per-sensor read/calibration logic (soil, sound, water, gas, heating, electricity/DLMS)
2	RS-485 bus framing, bridge polling cycle, leaf response handling
3	ESP32 platform primitives: task watchdog registration, brownout detector interaction, heap allocation patterns, boot-time WiFi state machine
4	Failure modes introduced *by* the fixes from passes 1–3 (e.g., a new retry loop's own worst-case resource use)

Table 1: The four audit passes and their scope.

Pass 4 is methodologically the most interesting: it treats every previous fix as a new piece of untrusted code subject to the same scrutiny as the code it replaced, rather than assuming a fix is correct because it addresses the defect it was written for. Two of the defects reported in § 3 (a heap-fragmentation risk in a WiFi retry loop, and an unbounded memory read in an HTTP response handler) were found during pass 4, in code written during passes 2–3 of the same audit.

2.2 Fault taxonomy

We group the sixteen defects found into four classes.

Class A — unbounded blocking without watchdog protection. A code path that can block for an unbounded or very long duration, in a build configuration where the ESP32 task watchdog timer (TWDT) was never registered for that task. Seven of the fleet's 32 build configurations — primarily bring-up/diagnostic firmware images, but including at least one passively field-deployed display mode — fell into this class: any hang in those paths required a manual power cycle to recover from, with no automatic reset.

Class B — sensor fault masking. A sensor read path that, on a disconnected probe, a shorted input, or an otherwise-invalid physical signal, returns a numerically plausible value with a "valid" flag set, rather than flagging the reading as faulty. This is the most operationally dangerous class, because the failure is invisible at every downstream layer: the device reports it, the backend stores it, and the dashboard renders it as if it were real.

Class C — unbounded memory allocation. A code path whose memory allocation size is a function of untrusted input (a network response, a sensor burst) with no upper bound enforced *before* the allocation occurs, as distinct from an upper bound enforced only after the fact.

Class D — boot-time race conditions. A one-shot readiness check performed once at boot, whose negative result is treated as permanent for the remainder of the run, even though the underlying condition (a peripheral's power rail settling, a WiFi access point finishing its own boot after a shared power event) is transient and would resolve within seconds if only the check were retried.

2.3 Validity-check formulas

Three sensor drivers required a closed-form validity check derived from their own calibration, rather than a generic range clamp.

Capacitive soil moisture. The raw analog-to-digital reading r is converted to a percentage via a two-point calibration $\left(r_{\text{dry}}, r_{\text{wet}}\right)$, measured in air and in water respectively:

$$\hat{p} \;=\; \frac{r_{\text{dry}} - r}{r_{\text{dry}} - r_{\text{wet}}} \times 100, \qquad p \;=\; \text{clamp}(\hat{p},\, 0,\, 100)\,. \tag{1}$$

Two failure modes are latent in Eq. (1) alone. First, if $r_{\text{dry}} = r_{\text{wet}}$ (a calibration error), the expression is $0/0$, and IEEE 754 division produces `NaN`; because `clamp` in the runtime's standard library is comparison-based and every comparison against `NaN` is false, `NaN` passes through *unclamped* and can reach a downstream JSON serializer as a syntactically invalid token. Second, and more consequentially: a disconnected or shorted probe does not fail to produce a reading — it produces r pinned at or near one ADC rail (0 or the converter's full scale), and Eq. (1) maps that directly to a perfectly plausible p near 0% or 100%, indistinguishable from a genuinely dry or saturated substrate. We therefore add an explicit fault predicate, independent of the percentage computation:

$$\text{fault}(r) \;=\; \begin{cases} \text{true} & r \le \min(r_{\text{dry}}, r_{\text{wet}}) - m \\ \text{true} & r \ge \max(r_{\text{dry}}, r_{\text{wet}}) + m \\ \text{false} & \text{otherwise,} \end{cases} \tag{2}$$

with margin $m = \max\!\left(0.1\,|r_{\text{dry}} - r_{\text{wet}}|,\; m_{\min}\right)$ for a small absolute floor $m_{\min}$ — a correctly wired sensor should never read materially beyond its own two calibration endpoints, so exceeding them by more than measurement noise is a stronger disconnect signal than a merely extreme, still-in-range percentage.

Current-loop pressure transducers (water, gas). A 4–20 mA transmitter's loop current is recovered from a shunt-resistor voltage V (itself an oversampled ADC mean) and mapped to pressure with a two-stage linear transform:

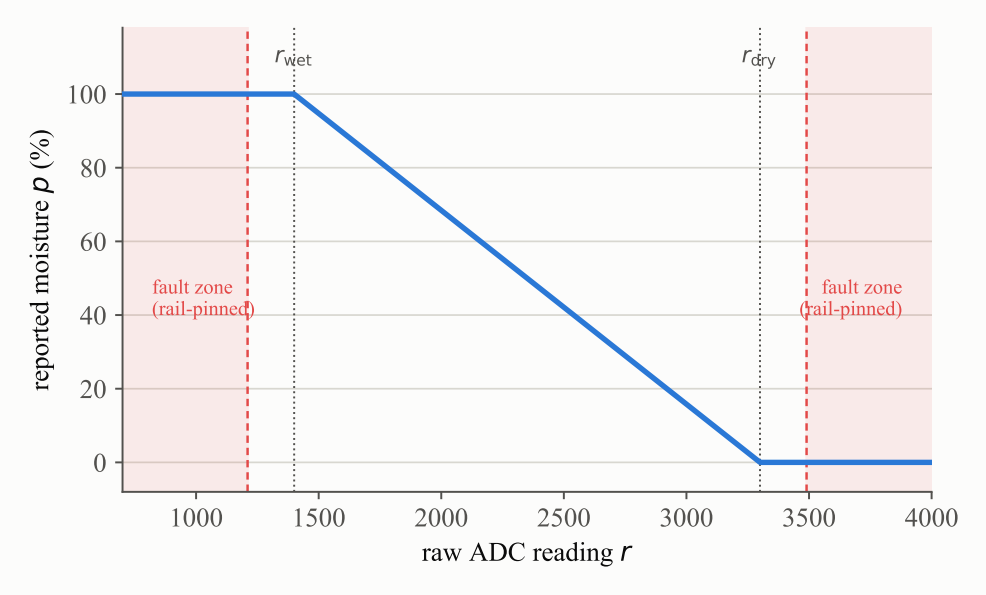

Figure 1: Eq. (1) (solid line) and the fault predicate of Eq. (2) (shaded bands) for $r_{\text{dry}} = 3300$, $r_{\text{wet}} = 1400$. A rail-pinned reading in either shaded region is flagged as a sensor fault rather than reported as an extreme-but-plausible percentage.

$$I = \frac{V}{R_{\text{shunt}}} \times 1000 \quad \text{(mA)} \tag{3}$$

$$P = \text{clamp}\left(\frac{I - I_{\min}}{I_{\max} - I_{\min}} \times P_{\max},\ 0,\ P_{\max} \right) \tag{4}$$

with the transmitter's rated $I_{\min} = 4\,\text{mA}$, $I_{\max} = 20\,\text{mA}$. Values of I outside a wider tolerance band $[I_{\text{err,lo}}, I_{\text{err,hi}}]$ (e.g., $I \approx 0$, indicating a broken loop) are flagged as faulty *before* Eq. (4) is applied, rather than silently clamping to the pressure range's boundary as Eq. (4) alone would do.

Exponential smoothing. Several channels apply a first-order IIR low-pass (an exponential moving average) to a noisy but otherwise valid reading before display or transmission:

$$\hat{x}_t \;=\; \alpha\, x_t + (1 - \alpha)\, \hat{x}_{t-1}, \qquad \alpha \in (0, 1]. \tag{5}$$

A subtlety we highlight for reproducibility: Eq. (5) must be seeded with $\hat{x}_0 = x_0$ (not $\hat{x}_0 = 0$) on the first genuinely valid reading following a boot or a fault-recovery event, or the filter's own startup transient is visually indistinguishable from a real, slowly-rising physical signal for several sampling intervals.

2.4 Bounded-latency design for oversampled reads

An oversampled ADC read of k samples over an intermittently — not permanently — unresponsive I2C bus can, in the worst case, spend up to the bus driver's own per-transaction timeout τ on every sample:

$$T_{\text{read}}^{\max} \;=\; k\,\tau. \tag{6}$$

For $k = 16$ and $\tau = 50\,\text{ms}$ (a value chosen specifically so a genuinely wedged bus cannot hang the read forever), Eq. (6) gives an bound of 800 ms for a *single* channel read — large enough, across a two-channel sensor, to noticeably stall a control loop that also owes service to a watchdog and a network

stack on a fixed budget. We replace the fixed sample count with an early-exit rule that still guarantees a minimum sample count $k_{\min}$ for measurement quality under normal conditions:

$$\text{stop after sample } i \text{ if } \; i \geq k_{\min} \;\wedge\; \sum_{j=1}^{i} t_j > B, \tag{7}$$

where t_j is the wall-clock cost of sample j and B is a time budget chosen well below $k\tau$. With $k_{\min} = k/2 = 8$ and $B = 150\,\text{ms}$, the realized worst case is $k_{\min}\tau = 400\,\text{ms}$ — a factor-of-two reduction — while a healthy bus (every $t_j \ll \tau$) is unaffected, since the budget is never reached before all k samples complete.

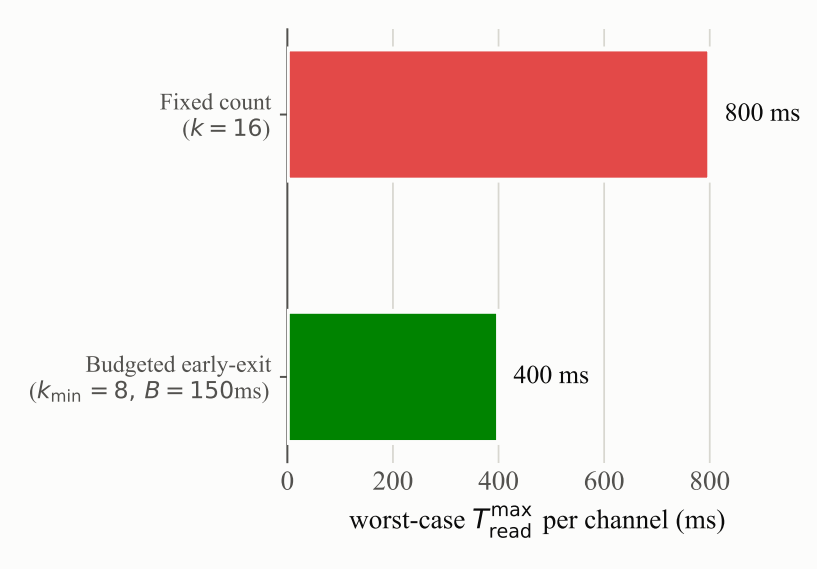

Figure 2: Worst-case per-channel read latency under Eq. (6) (fixed $k = 16$ samples) versus the budgeted early-exit rule of Eq. (7) ($k_{\min} = 8$, $B = 150\,\text{ms}$) — a factor-of-two reduction in the worst case, with no change to normal-case sample count or latency.

2.5 A conservatively reverted fix: receive-side frame checksums

The metering protocol described in the companion paper's § 2.3 computes a 16-bit checksum over every transmitted frame but — as originally implemented — never verified that checksum on *receipt*. On a physically noisy RS-485 line, a corrupted-but-structurally-plausible frame (correct start/end delimiters, corrupted payload) would be parsed as trusted data, silently producing a numerically wrong — not obviously invalid — meter reading. We implemented and compile-verified a receive-side check mirroring the transmit-side checksum computation.

We then reverted it. The reasoning, made explicit here because we consider it methodologically the most important result in this section: the checksummed byte range on receive must *exactly* match the byte range checksummed on transmit, and this equivalence had only been verified by code inspection and successful compilation — never against a real meter, over a real cable, with real electrical noise. An off-by-one in the checksummed range would not manifest as a compile error, nor as an obviously wrong value; it would manifest as *every* real-hardware read being silently rejected as "corrupted," which is a strictly worse outcome for the platform's core function (metering) than the noise-corruption risk the check was meant to catch. Absent a hardware-in-the-loop test able to distinguish these two outcomes before a fleet-wide deployment, we judged the fix's downside risk to dominate its upside, and reverted to the pre-existing (unverified-on-receive, but field-proven) behavior. The check remains implemented in version control and is a candidate for re-introduction once bench-validated against a physical meter.

3 Results

Table 2 summarizes the sixteen defects by class, cross-referenced against the layer of the system in which each was found. All fixes except the one described in § 2.5 remain in the deployed firmware at the time of writing; every fix was verified by a full recompilation of every affected PlatformIO build target (up to fifteen per change) before being considered complete, with zero regressions introduced across the audit.

Class	Count	Representative instance
A — unbounded blocking, no watchdog	4	7 of 32 build configurations never registered with the task watchdog; one bring-up firmware path contained a genuine unconditional infinite loop on missing hardware.
B — sensor fault masking	5	A disconnected soil probe reported a plausible 0% reading indefinitely; a disconnected microphone's boot-time calibration silently substituted a fixed baseline, reporting "quiet room" forever.
C — unbounded allocation	2	A chunked (unknown-length) HTTP response was fully buffered *before* a size limit was checked, defeating the limit's purpose.
D — boot-time race conditions	3	An I2C display driver's one-shot boot probe, if it lost a timing race against the display module's own power-up, left the screen blank for the entire session with no retry.
Other (protocol/timing/config)	2	A Wi-Fi captive-configuration portal disabled the station radio for its full duration with no bounded fallback; a bridge polling cycle's internal timeout exceeded its own nominal reply window.

Table 2: Sixteen defects found across four audit passes, by class.

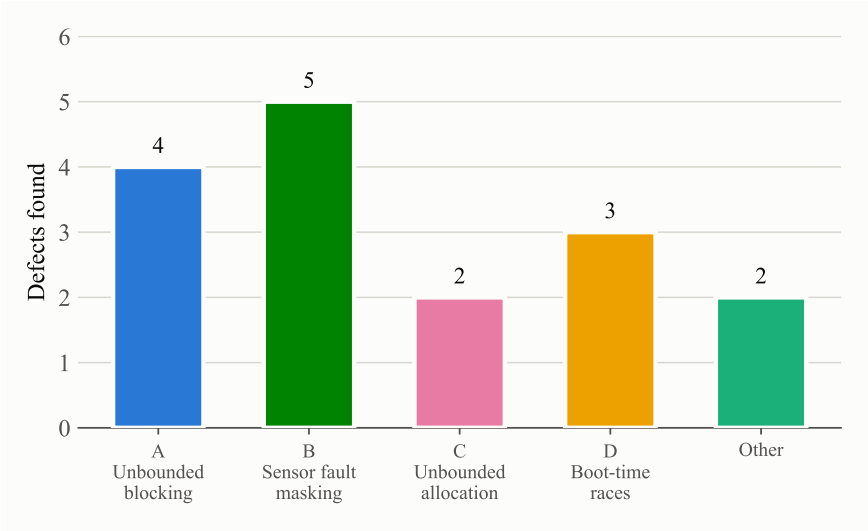

Figure 3: Defect counts by class (Table 2). Class B (sensor fault masking) is the largest single category despite none of its instances being able to hang or crash the device — see the discussion of silent-versus-loud failure in § 4.

The Class D instance is worth calling out as a second-order finding: it was *introduced* by an earlier, well-intentioned fix in the same audit (making a display driver's "is the hardware present" flag honestly reflect a failed probe, rather than always optimistically reporting success) that inadvertently removed the only code path that would have retried the probe. This is a direct instance of the audit's own pass-4

methodology (§ 2.1) catching a defect in its own earlier output, and we regard it as evidence for treating every fix as subject to the same scrutiny as the code it replaces, not as an exception to it.

4 DISCUSSION

Silent correctness failures dominate. Of the sixteen defects, the five in Class B (sensor fault masking) are, in our judgment, the most consequential despite none of them being able to crash or hang the device: a system that is up, responsive, and reporting numbers that are simply wrong is harder to detect and diagnose in production than one that is visibly down. We suggest this asymmetry — crashes are self-reporting, silent wrong-answers are not — as a general prioritization heuristic for embedded reliability audits operating under limited time: audit for Class B before Class A, even though Class A defects (hangs, watchdog gaps) are often easier to find via static inspection.

A compile matrix is a necessary but not sufficient regression gate. Recompiling all affected build targets after every change caught zero silent breakages across sixteen fixes in this audit — itself a useful negative result, suggesting the codebase's build configuration matrix was already reasonably decoupled per sensor type. However, a successful compile says nothing about runtime correctness on the specific hardware a firmware image targets, as the reverted checksum fix in § 2.5 illustrates directly: that fix compiled cleanly across every relevant target and was, by every static-inspection criterion available to us, apparently correct. We regard hardware-in-the-loop testing – not yet performed for this fleet at the time of writing – as the single highest-value remaining verification step, and recommend it explicitly be budgeted as a distinct phase from code review in future firmware reliability work, rather than treated as an optional follow-up.

Limitations. This audit was conducted entirely through static code review and compile-time verification; no physical fault injection (e.g., deliberately disconnecting a sensor mid-operation, introducing line noise on the RS-485 bus, or brown-out testing under a marginal power supply) was performed. Several of the findings reported here — the soil-moisture and pressure-transducer fault predicates in particular — were validated only against the sensors' documented calibration behavior, not against a population of physical units exhibiting real failure modes. We consider the results reported here a necessary precondition for, rather than a substitute for, field validation.

Future work. Three items follow directly from this audit: (1) hardware-in-the-loop validation of the reverted receive-side checksum (§ 2.5), after which it should be re-introduced if confirmed safe; (2) a brown-out and power-supply-margin test campaign, since the watchdog/reliability work in this audit was necessarily reactive to a documented but unmeasured platform-level brown-out risk rather than addressing it at the root cause; (3) extending the compile-matrix methodology (§ 2.1) into an automated regression suite triggered on every firmware change, rather than the manually-invoked verification process used during this audit.